

บทที่ 6

การออกแบบ

6.1 แนวความคิดในการออกแบบ

การออกแบบชุดต้นแบบการเข้ารหัสด้วยฮาร์ดแวร์ จะเริ่มด้วยการมองภาพรวมของการทำงานทั้งหมดของระบบ หลังจากนั้นจะทำการศึกษาทฤษฎีต่าง ๆ ที่เกี่ยวข้องจนเข้าใจความต้องการทั้งหมดของระบบแล้ว เราจึงเริ่มทำการออกแบบระบบรวมทั้งหมด โดยในการออกแบบนั้น จะทำการแบ่งระบบออกเป็นส่วนย่อย ๆ โดยในแต่ละส่วนจะถูกออกแบบให้มีส่วนติดต่อที่เหมือนกับส่วนติดต่อมาตรฐานให้มากที่สุด เพื่อความสามารถในการใช้งานร่วมกันได้กับระบบที่มีอยู่แล้วในปัจจุบัน และความสามารถในการนำเอาส่วนต่าง ๆ กลับมาใช้ได้อีก

6.2 ภาพรวมในการออกแบบ

ในการออกแบบ เราจะยึดเอาการติดต่อระหว่างส่วนต่าง ๆ เป็นหลัก ซึ่งทำให้เราแบ่งส่วนต่าง ๆ ออกมาเป็น 3 ส่วนใหญ่ ๆ คือ ส่วนของอุปกรณ์ฝังฮาร์ดแวร์ ซอฟต์แวร์ดีไวซ์ไดรเวอร์ (Software Device Driver) และซอฟต์แวร์ โดยฮาร์ดแวร์และซอฟต์แวร์ทั้ง 2 ส่วน จะทำงานร่วมกันโดยผ่านบริการของระบบปฏิบัติการ (OS Services) ส่วนของฮาร์ดแวร์จะทำการติดต่อกับพีซีไอ (PCI Bus) โดยมีการ์ดพีซีไอ (PCI Card) เป็นส่วนหน้าในการเชื่อมต่อของฝังฮาร์ดแวร์ โดยการทำงานส่วนใหญ่ในฝั่งนี้จะขึ้นอยู่กับบอร์ดเอฟพีจีเอ (FPGA Board) เป็นหลัก ซึ่งการเข้ารหัสของข้อมูลจะถูกกระทำในส่วนของฝังฮาร์ดแวร์นี้ รวมถึงการควบคุมการติดต่อกับพีซีไอบัสด้วย ในรุ่นแรกนี้ เราจะยังไม่ได้ใช้ความสามารถทั้งหมดของบัส เนื่องจากข้อจำกัดของเวลา และขนาดพื้นที่บอร์ดเอฟพีจีเอของเนื่องจากการพัฒนาในส่วนของฮาร์ดแวร์จะต้องใช้เวลานานกว่าการพัฒนาในส่วนซอฟต์แวร์

สำหรับส่วนของการเข้ารหัสที่อยู่ในฝั่งฮาร์ดแวร์นั้น ประกอบด้วยสามฟังก์ชันคือในส่วนของการทำการสุ่มเลขเทียมโดยใช้อัลกอริทึม LFSR, การทำแฮชอัลกอริทึมโดยใช้อัลกอริทึม SHA1, และการเข้าและถอดรหัสลับซึ่งเป็นแบบ DES และ 3DES

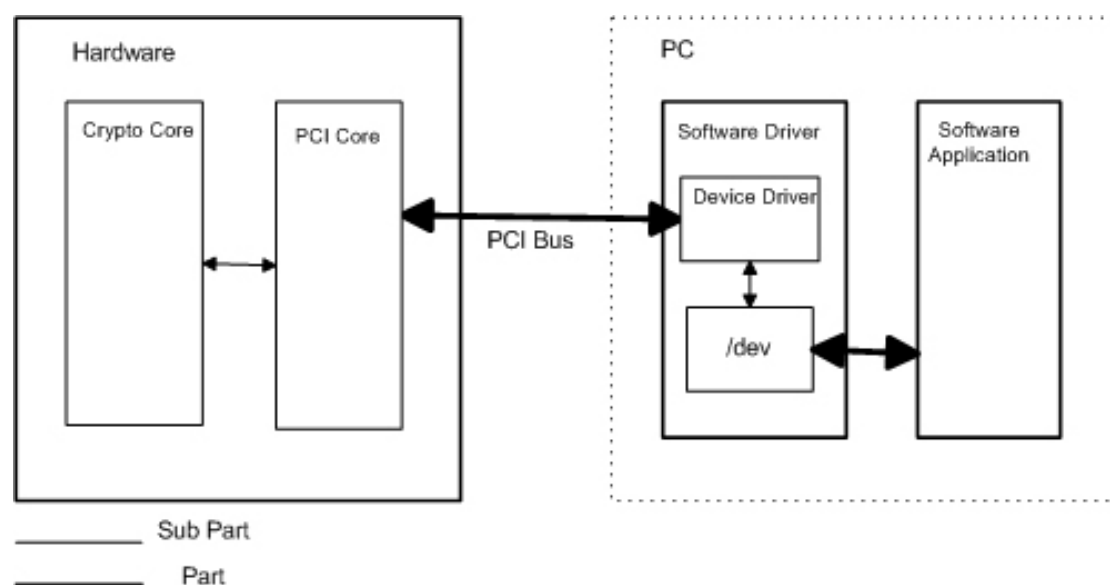
ส่วนของซอฟต์แวร์ดีไวซ์ไดรเวอร์ จะทำงานโดยฝังตัวเข้ากับบริการของระบบปฏิบัติการ โดยจะอาศัยบริการหลัก (กลาง) ของระบบปฏิบัติการทำงาน และสร้างบริการของระบบปฏิบัติการขึ้นมาใหม่ชุดหนึ่ง เพื่อใช้ในการเข้าถึงอุปกรณ์ฝังฮาร์ดแวร์ รวมถึงเพื่อให้บริการแก่โปรแกรมใช้งานทั่วไปให้สามารถเรียกใช้งานได้ด้วย สำหรับระบบปฏิบัติการที่เลือกมาใช้จะเป็น

ระบบปฏิบัติการ Linux ที่ Kernel รุ่น 2.6.0 มาตรฐาน สาเหตุที่เลือกนำระบบปฏิบัติการตัวนี้มาใช้ เพราะเห็นถึงความง่ายในการพัฒนาและรวมถึงชุดพัฒนาที่สามารถหามาใช้ได้ง่าย

อีกส่วนหนึ่งของซอฟต์แวร์ คือ เราจะทำการพัฒนาโปรแกรมที่มาทดสอบการทำงานทั้งสามส่วนให้ถูกต้องตามมาตรฐาน สำหรับการทำงานระหว่างซอฟต์แวร์ทดสอบกับส่วนของซอฟต์แวร์ดีไวซ์ไครเวอร์นั้น ยังต้องติดต่อถึงกันและกันโดยผ่านส่วนติดต่อมาตรฐานของระบบปฏิบัติการอีกที รวมถึงในด้านสถานะแวดล้อมของการใช้งาน โปรแกรมดีไวซ์ไครเวอร์จะรันในเคอร์เนลโหมด ส่วนโปรแกรมวิพีเอ็นจะรันในยูสเซอร์โหมด

6.3 การทำงานร่วมกันในแต่ละส่วน

ส่วนต่างๆของชุดต้นแบบนี้จะติดต่อซึ่งกันและกันตามรูปด้านล่าง ในระบบของชุดต้นแบบจะไม่ มีการติดต่อถึงกันโดยตรงระหว่างส่วนต่าง ๆ นอกเหนือจากที่แสดงไว้ในรูปข้างล่าง สำหรับส่วนฝั่งฮาร์ดแวร์จะทำการติดต่อกับระบบปฏิบัติการผ่านทางพีซีไอบัส(PCI Bus) เรียกใช้งานดีไวซ์ไครเวอร์ผ่านทางไฟล์โอเปอเรชัน(File Operation) โดยการรับส่งไฟล์ผ่านฟังก์ชันอ่าน (Write) และเขียน(Read) ควบคุมการทำงานต่างผ่านฟังก์ชันไอโอคอนโทรล(I/O Control)



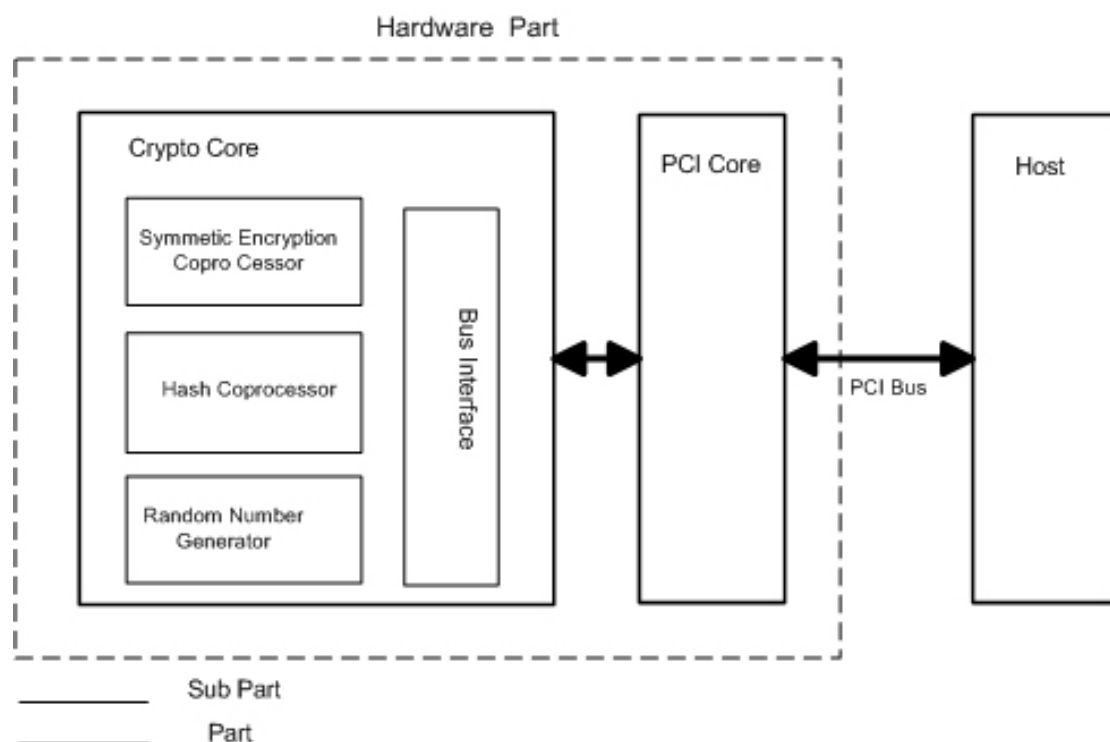
รูปที่ 6.1 แสดงการทำงานร่วมกันระหว่างแต่ละส่วนของระบบ

6.4 การทำงานในแต่ละส่วนย่อย

6.4.1 การออกแบบการทำงานส่วนของฮาร์ดแวร์

เนื่องจากการทำงานโดยใช้ซอฟต์แวร์นั้น ยังมีขีดจำกัดในเรื่องความเร็วในการส่งข้อมูล และความยืดหยุ่นในการนำไปประยุกต์ใช้งานในสภาพการทำงานปกติ ดังนั้นโครงการนี้จึงได้ออกแบบการทำงานในส่วนที่ต้องมีการใช้การประมวลผลมากใช้ฮาร์ดแวร์เข้ามาช่วยแก้ปัญหาในส่วนนี้ และต้นแบบในโครงการนี้ยังสามารถนำไปพัฒนาต่อให้ใช้งานได้เป็นอิสระโดยไม่ต้องพึ่งพาเครื่องคอมพิวเตอร์ส่วนบุคคลในการทำงานได้ด้วย

ในส่วนออกแบบอุปกรณ์ส่วนฮาร์ดแวร์เพื่อให้เป็นมาตรฐานและง่ายในการไปพัฒนาต่อจึงได้แบ่งออกเป็นสองส่วนคือ ส่วนการติดต่อกับพีซีไอบัส(PCI Core) และ ส่วนที่รวมการทำงานในด้านการเข้ารหัสถอดรหัสไว้ (Crypto Core) ตามรูปข้างล่าง



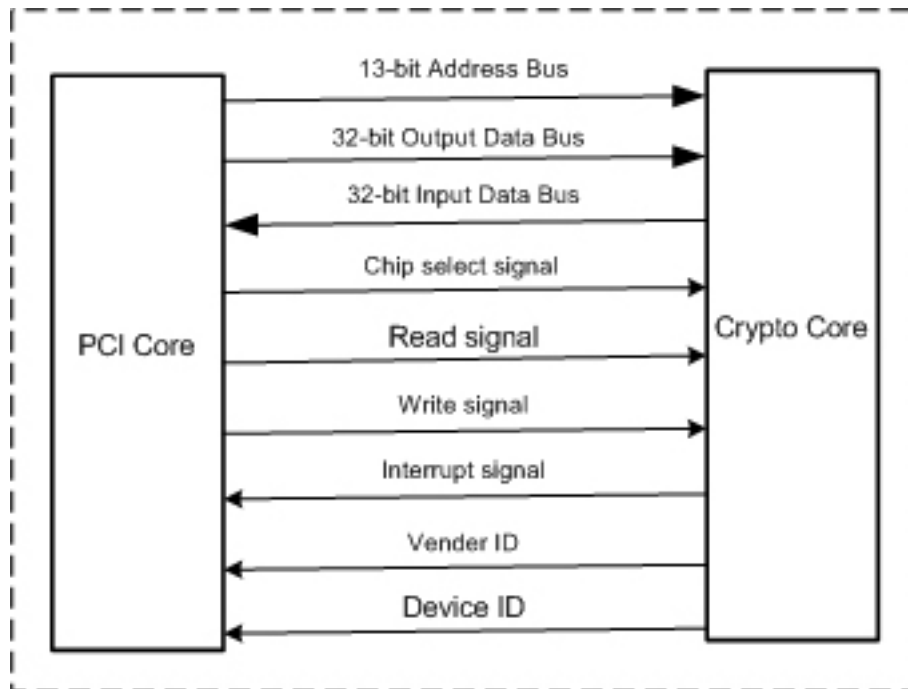
รูปที่ 6.2 ระบบย่อยส่วนต่างๆของฮาร์ดแวร์

และสำหรับส่วนคริปโตคอร์(Crypto Core) ภายในประกอบด้วยส่วนย่อยๆอีกคือ

- DES
- SHA1
- LFSR128
- Bus Interface
- Device ID/Vender ID

1. ชุดเชื่อมต่อพีซีไอ (PCI Core)

สำหรับ PCI Core ทำหน้าที่เชื่อมต่อการทำงานระหว่างพีซีไอ (PCI Bus) กับชุดเข้ารหัส โดยที่ Local Bus เป็นการเชื่อมต่อแบบ Memory Interface ซึ่งใช้สัญญาณ Address Bus, Data Bus, Read, Write และ Chip Select เป็นหลักในการเชื่อมต่อ มีลักษณะการเชื่อมต่อดังรูปข้างล่างนี้



รูปที่ 6.3 สัญญาณ Local Bus

- Address Bus 13 บิต (lbaddr[14:2]): เป็นสัญญาณ address bus ของ PCI.
- Input Data Bus 32 บิต (lbdatain[31:0])
- Output Data Bus 32 บิต (lbdataout[31:0])
- สัญญาณ Read, ทำงานแบบ active low (lbrdb)
- สัญญาณ Write, ทำงานแบบ active low (lbwrb)
- สัญญาณ Chip Select, ทำงานแบบ active low (lbcsb)
- สัญญาณ Interrupt, ทำงานแบบ active high (lbint)
- สัญญาณ Vender ID
- สัญญาณ Device ID

2. ชุดเข้ารหัส (Crypto Core)

สำหรับ Crypto Core ทำหน้าที่เชื่อมต่อการทำงานระหว่าง Local Bus ที่ได้รับจาก PCI Core แล้วส่งสัญญาณให้กับชุดเข้ารหัสภายใน สำหรับสัญญาณที่เชื่อมต่อ PCI core มีดังนี้

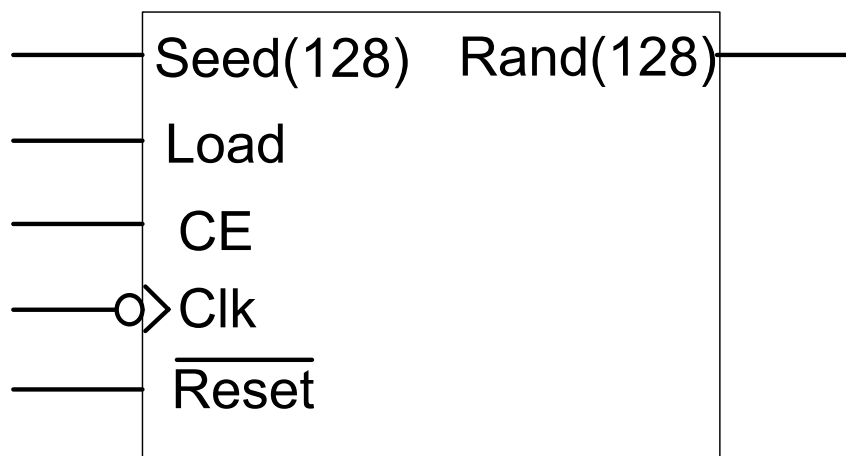
- Address Bus 13 บิต (lbaddr[14:2]): เป็นสัญญาณ address bus ของ PCI.
- Input Data Bus 32 บิต (lbdatain[31:0])
- Output Data Bus 32 บิต (lbdataout[31:0])
- สัญญาณ Read, ทำงานแบบ active low (lbrdb)
- สัญญาณ Write, ทำงานแบบ active low (lbwrdb)
- สัญญาณ Chip Select, ทำงานแบบ active low (lbcsb)
- สัญญาณ Interrupt, ทำงานแบบ active high (lbint)
- สัญญาณ Vender ID
- สัญญาณ Device ID

ภายใน Crypto Core ประกอบด้วยส่วนต่างๆดังนี้

- DES
- SHA1
- LFSR128
- Bus Interface
- Device ID/Vender ID

3. Linear Feedback Shift Register (LFSR)

เป็น Pseudo Random Number Generator ขนาด 128 bits ที่ใช้เวลา 1 Clock เพื่อให้ได้ค่าเลขสุ่มออกมา



รูปที่ 6.4 สัญญาณต่างๆของ LFSR

ตารางที่ 6.1 ตารางแสดงสัญญาณต่างๆของ LFSR

pin	direction	description
Seed (32)	input	ขนาด 32 bits ใช้สำหรับป้อนค่าเริ่มต้นให้กับ LFSR
Load	input	'1' เพื่อทำการป้อนค่าเริ่มต้นให้กับ LFSR
CE	input	'1' เพื่อกำหนดให้ LFSR ทำการสร้างเลขสุ่มเลขใหม่ขึ้นมา
Reset	input	'0' เป็นการเริ่มต้นเพื่อกำหนดค่า หรือ เริ่มต้นการทำงานใหม่
Clk	input	สัญญาณนาฬิกาเพื่อให้งานทำงานแบบ synchronous
Rand (32)	Output	สร้างเลขสุ่มขนาด 32 bits เมื่อมีการทำงานที่เรียบร้อย

4. Data Encryption Standard (DES)

ใช้เวลา 16 Clock ในการเข้ารหัสหรือถอดรหัสลับ โดย Key และ Input ขนาด 64 bits



รูปที่ 6.5 แสดงสัญญาณ DES

ตารางที่ 6.2 ตารางแสดงสัญญาณต่างๆของ DES

pin	direction	description
DataIn (64)	input	สัญญาณขนาด 64 bits ส่งเข้า เพื่อเข้ารหัสหรือถอดรหัส
Key (64)	input	ส่ง Key ขนาด 64 bits
Decipher	input	'0' เพื่อต้องการเข้ารหัส '1' เพื่อถอดรหัส
Ds	input	'1' เพื่อเริ่มการเข้ารหัสหรือถอดรหัส
Reset	input	'0' เป็นการเริ่มต้นเพื่อกำหนดค่า หรือ เริ่มต้นการทำงานใหม่
Clk	input	สัญญาณนาฬิกาเพื่อให้งานทำงานแบบ synchronous
DataOut (64)	Output	เมื่อมีการเข้ารหัสหรือถอดรหัสแล้วจะได้ผลลัพธ์ขนาด 64
Ready	Output	เมื่อมีการเข้ารหัสหรือถอดรหัสเรียบร้อยแล้วจะมีสัญญาณ '1' นาน 1 Clock

DataIn(64)

Key(64)

Decipher

Ds

5. Secure Hash Algorithm (SHA1)

ใช้เวลาทั้งหมด 80 Clock ในการสร้าง Digest สามารถเข้ารหัสข้อความแล้วได้ Digest

ขนาด 160 bits



รูปที่ 6.6 สัญญาณต่างๆของ SHA1

ตารางที่ 6.3 ตารางแสดงสัญญาณต่างๆของ SHA1

pin	direction	description
Message (512)	input	ข้อความขนาด 512 bits ในการหา Digest
Message Valid	input	‘1’ เป็นการเริ่มการหา Digest
Reset	input	‘0’ เป็นการเริ่มต้นเพื่อกำหนดค่าหรือเริ่มต้นการทำงานใหม่
Clk	input	สัญญาณนาฬิกาเพื่อให้งานเป็น synchronous
Digest (160)	Output	Digest ขนาด 160 bits เมื่อกระบวนการหา Digest เรียบร้อย
Digest Valid	Output	เมื่อกระบวนการหา Digest เรียบร้อยแล้วสัญญาณนี้จะ เป็น ‘1’ นาน 1 clock

6. Device ID

ทำหน้าที่เพียงส่งค่า Device ID และ Vender ID ให้กับ Crypto Core เท่านั้น ตอนนี้ให้ค่าคงที่ไว้ดังนี้

- Vender ID -> F0F0
- Device ID -> F0F0

7. การนำไปใช้

เราใช้การ์ด PCI อเนกประสงค์ TruePCI สองใบสำหรับประจุ Linear Feedback Shift Register, Data Encryption Standard, Secure Hash Algorithm จึงได้แบ่งออกเป็นสองกลุ่มดังนี้

- Secure Hash Algorithm
- Linear Feedback Shift Register, Data Encryption Standard

ได้ออกแบบการทำงานเป็นแบบ Memory Interface ผ่านทางสัญญาณ Read, Write, Chip Select, Address, Data In, Data Out ในการส่งข้อมูลต่างๆต้องส่งตาม Address

การ์ดใบที่ 1 (Secure Hash Algorithm)

ภายในบรรจุด้วย Secure Hash Algorithm มีตำแหน่ง Address และการเข้าถึงค่าต่างๆดังนี้

ตารางที่ 6.4 ตารางแสดงตำแหน่ง Address ของการ์ดใบที่ 1

Pin	Address	Direction
0004	Message 0	Input
0008	Message 1	Input
000c	Message 2	Input
0010	Message 3	Input
0014	Message 5	Input
0018	Message 6	Input
0020	Message 7	Input
0024	Message 8	Input
0028	Message 9	Input
002c	Message 10	Input
0030	Message 11	Input
0034	Message 12	Input
0038	Message 13	Input
003c	Message 14	Input
0040	Message 15	Input
0044	Message Valid	Input
0048	Reset	Input
0050	Digest E	Output

ตารางที่ 6.4 (ต่อ)

0054	Digest D	Output
0058	Digest C	Output
005c	Digest B	Output
0060	Digest A	Output
0064	Digest Valid	Output

การ์ดไบนารี 2 (Linear Feedback Shift Register, Data Encryption Standard)

ภายในบรรจุด้วย Linear Feedback Shift Register, Data Encryption Standard มีตำแหน่ง Address และการเข้าถึงค่าต่างๆดังนี้

Data Encryption Standard

ตารางที่ 6.5 ตารางแสดงตำแหน่ง Address ของการ์ดไบนารี 2 ในส่วนของ DES

Pin	Address	Direction
0004	Data In (Low)	Input
0008	Data In (High)	Input
000c	Key (Low)	Input
0010	Key (High)	Input
0014	Decipher	Input
0018	DS	Input
0020	Data Out (Low)	Output
0024	Data Out (High)	Output
0028	Ready	Output

Linear Feedback Shift Register

ตารางที่ 6.6 ตารางแสดงตำแหน่ง Address ของการ์ดไบนารี 2 ในส่วนของ LFSR

Pin	Address	Direction
0104	Seed (31 downto 0)	Input
0108	Seed (63 downto 32)	Input

ตารางที่ 6.6 (ต่อ)

010c	Seed (95 downto 64)	Input
0110	Seed (127 downto 96)	Input
0114	Load	Input
0118	CE	Input
0120	Rand (31 downto 0)	Output
0124	Rand (63 downto 32)	Output
0128	Rand (95 downto 64)	Output
012c	Rand (127 downto 96)	Output

6.4.2 การออกแบบการทำงานของซอฟต์แวร์ดีไวซ์ไครเวอร์

ส่วนของซอฟต์แวร์ดีไวซ์ไครเวอร์นั้นถูกโปรแกรมออกมาเพื่อทำหน้าที่ 2 อย่าง คือ การควบคุมข้อ กำหนดในการใช้งานอุปกรณ์ และสร้างบริการสำหรับการใช้งานอุปกรณ์จากโปรแกรมผู้ใช้งานทั่วไป

แต่ซอฟต์แวร์ดีไวซ์ไครเวอร์ยังแบ่งส่วนติดต่อได้เป็น 2 ฟังก์ชัน คือ ส่วนติดต่อกับอุปกรณ์ และส่วนติดต่อ ผู้ใช้งาน ในส่วนติดต่อกับอุปกรณ์นั้น เราจะให้บริการในระดับล่างของระบบปฏิบัติการเพื่อจองทรัพยากร และเพื่อรับส่งข้อมูลกับอุปกรณ์ฮาร์ดแวร์ เช่น การจองพอร์ตแอดเดรสของอุปกรณ์ และการส่งข้อมูลออกทางพอร์ต เป็นต้น สำหรับส่วนติดต่อผู้ใช้งานนั้น จะให้บริการในระดับบนเพื่อเพิ่มบริการของตัวเองเข้ากับบริการที่มีอยู่ของระบบปฏิบัติการ การทำงานหลัก ๆ ของไครเวอร์ คือ การควบคุมการรับส่งข้อมูลระหว่าง ส่วนติดต่อทั้ง 2 ฟังก์ชัน

สำหรับส่วนติดต่อกับผู้ใช้งานนั้น เราจะเลือกใช้การให้บริการโดยมองอุปกรณ์เสมือนเป็นแฟ้มข้อมูลธรรมดา (Device File) ซึ่งในการใช้งานนั้น โปรแกรมฝั่งผู้ใช้งานสามารถเปิด ดีไวซ์ไฟล์ได้โดยผ่านทางฟังก์ชัน (Function) `open()` และสามารถรับส่งข้อมูลเพื่อทำการเข้ารหัสได้โดยผ่านทางฟังก์ชัน `read()` และ `write()` รวมถึงสามารถควบคุมการใช้งานอื่น ๆ ได้โดยผ่านทางฟังก์ชัน `ioctl()` ซึ่งในตัวติดต่อกับผู้ใช้งานนั้น จะให้บริการเฉพาะฟังก์ชันที่จำเป็นสำหรับการใช้งานในเบื้องต้นเท่านั้น โดยในซอฟต์แวร์ดีไวซ์ไครเวอร์ของเรานั้น จะรองรับฟังก์ชันตามการประกาศโครงสร้างต่อไปนี้

```
struct file_operations cryptochip_fops = {
    .owner = THIS_MODULE,
    .read = cryptochip_read,
    .write = cryptochip_write,
```

```

        .ioctl    =    cryptochip_ioctl,
        .open     =    cryptochip_open,
        .release  =    cryptochip_release,

};

```

การควบคุมการติดต่อสื่อสารนั้น เราจะต้องสามารถรับประกันความถูกต้องของข้อมูลได้ด้วย ซึ่งจะต้องรับประกันได้ว่า ถ้าโปรแกรมสามารถ write() หรือ read() ได้ โดยไม่ได้รับการแจ้งเตือน (return) ความผิดพลาด ดังนั้นข้อมูลที่ได้ทำการรับส่งหรือทำการประมวลผลกับดีไวซ์ใดเวอร์นั้น ก็จะต้องเป็นข้อมูลที่มีความถูกต้องด้วย ซึ่งสิ่งสำคัญสำหรับการนี้คือ การป้องกันการเรียกโปรแกรมซ้อนทับ โดยเฉพาะอย่างยิ่ง การเขียนทับที่ตำแหน่งเดียวกัน ซึ่งในประเด็นนี้ เราจึงแก้ปัญหาโดยใช้ซีมาฟอว์ (Semaphore) มาช่วยในการป้องกันการรันฟังก์ชันซ้อนทับกัน

ดีไวซ์ใดเวอร์นั้นได้ถูกออกแบบให้ประกอบด้วยส่วนของเฮดเดอร์ไฟล์และโปรแกรมไฟล์ ซึ่งส่วนของเฮดเดอร์ไฟล์จะเก็บค่าต่าง ๆ ที่เกี่ยวกับการ์ดไว้ ทำให้ง่ายในการปรับแต่ง มีไฟล์ชื่อว่า cryptocard.c และ cryptocard.h

6.4.3 การออกแบบการทำงานของซอฟต์แวร์แอปพลิเคชัน

ทำการออกแบบโปรแกรมแอปพลิเคชันเพื่อมาทดสอบการทำงานทั้ง 3 ส่วน คือ ส่วนของการทำการสุ่มเลขเทียมว่าสามารถทำงานได้จริงถูกต้องและมีประสิทธิภาพ ส่วนของการทำแฮชอัลกอริทึม ส่วนของการทำการเข้ารหัสลับและถอดรหัสลับ เพื่อให้ง่ายต่อการใช้งานและเป็นมาตรฐานจึงทำการออกแบบการใช้งานทั้ง 3 ส่วนผ่านทางไฟล์โอเปอเรชันซึ่งเป็นบริการของระบบปฏิบัติการ ส่งข้อมูลผ่านทางฟังก์ชัน read, write, และ ควบคุมการทำงานโดยใช้ I/O control ดังนี้

1. LFSR ขนาด 128 บิต ทำการส่งค่าตั้งต้น (seed) ผ่านฟังก์ชัน write และอ่านค่าเลขสุ่มเทียมผ่านฟังก์ชัน read
2. SHA1 ทำการส่งข้อความที่ต้องการผ่านทางฟังก์ชัน write และอ่านค่าเมสเสจไดเจสต์ที่ได้ออกมาผ่านฟังก์ชัน read
3. DES และ 3DES ทำการส่งข้อความที่ต้องการผ่านทางฟังก์ชัน write และอ่านค่าข้อความที่ทำการเข้ารหัสหรือถอดรหัสแล้วผ่านฟังก์ชัน read โดยเลือกโหมดและเลือกการเข้ารหัสหรือถอดรหัสผ่านทาง I/O Control

6.5 แนวคิดในการออกแบบชุดทดสอบ

สำหรับโครงการนี้ เราได้ออกแบบให้มีลักษณะเป็นมอดูลาร์ (Modular) ให้มากที่สุด ดังนั้นแต่ละส่วนจะมีความทำงานเป็นอิสระ ซึ่งจะช่วยให้เป็นอย่างมากในเรื่องของการนำเอาส่วนต่าง ๆ กลับมาใช้ได้อีกในงานต่อไป